

The Strong Survive: Feature Selection and Analysis

Sreeja Pagireddy and Svetlana Nosova

June 9, 2026

Introduction

In recent years, the datasets we use in machine learning tasks have become huge. We often deal with millions of instances and thousands or hundreds of thousands features. In such cases, machine learning models can easily overfit. In addition, as the number of features increases, a model training requires more resources. Even a small increase in number of features used for training may require companies to spend significantly more money. Feature selection helps to address these issues.

In this project, we analyze two algorithms for feature selection: forward selection and backward elimination. To compare feature subsets, we use one-nearest neighbor (1NN) classifier on synthetic and real datasets. This project combines supervised and unsupervised approaches to data analysis, so we also apply hierarchical clustering and display a dendrogram on a real dataset.

Feature Selection with Nearest Neighbor

Two synthetic datasets [1] were used to run forward and backward feature selection methods with 1-nearest neighbor classifier and leaving-one-out evaluation Sreeja created for Winter 2025 CS170 [2]. The smaller dataset, "CS170_Small_DataSet_1", contains 700 instances with 8 features, and the larger, "CS170_Large_DataSet_14", contains 3000 instances with 18 features.

The forward algorithm starts with an empty feature set and adds one feature at a time, choosing the feature that improves the accuracy better than other features do. The backward algorithm starts with the feature set that includes all original features, and then it removes one feature at a time such that it does not negatively affect the accuracy. Both algorithms are used to choose a subset of features that improves the chosen model's metric.

Dataset	Accuracy on All Features	Best Features Subset	Accuracy on Best Subset	Search Algorithm
Small Dataset	0.83	{2, 3}	0.96	Forward and Backward
Large Dataset	0.72	{9, 12}	0.97	Forward and Backward

Table 1: Feature selection results on synthetic datasets using 1-nearest neighbor with leave-one-out evaluation.

Table 1 represents results of using forward and backward algorithms with one-nearest neighbor on the two normalized datasets mentioned above. Running the classifier on all original features on the small dataset yielded accuracy of 0.83. Both forward and backward search algorithms identified features 2 and 3 as the best feature subset, which improved the accuracy to 0.96. On the large dataset, running the classifier on original features yielded an accuracy of 0.72. Both forward and backward methods identified features 9 and 12 as the best feature subset, which improved the accuracy of 0.97.

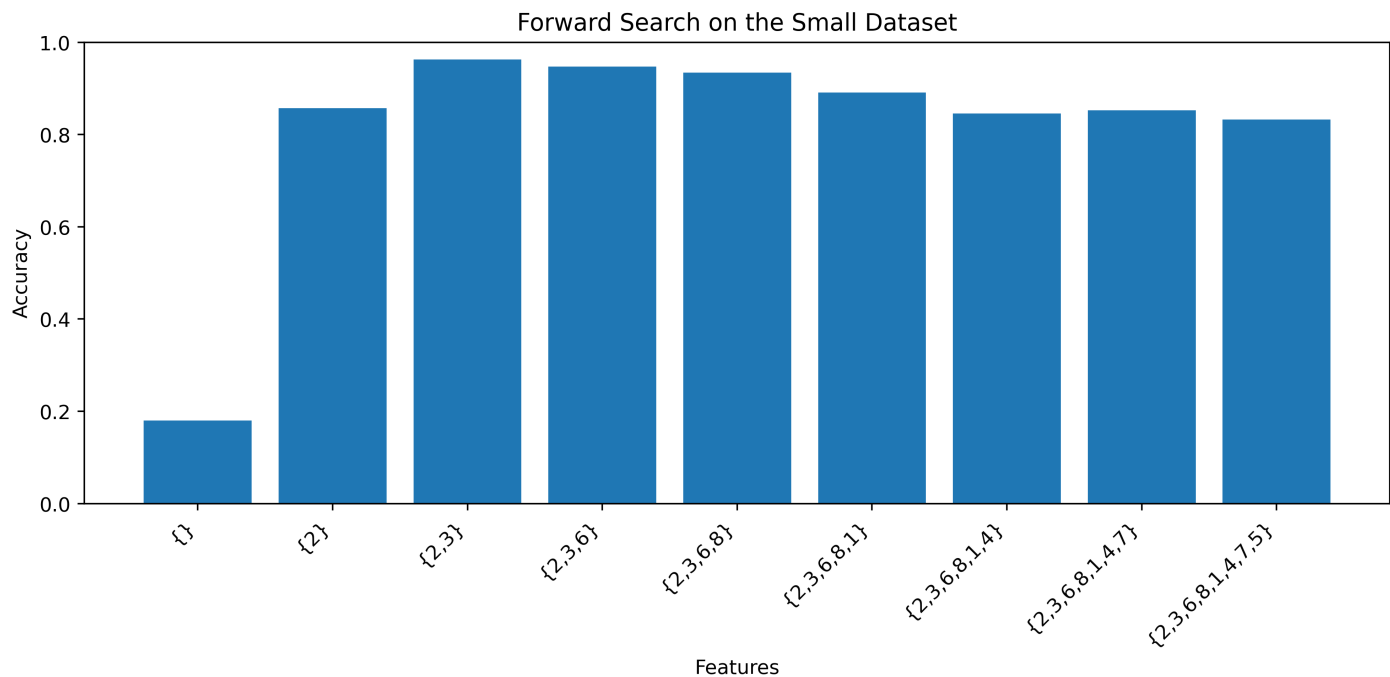


Figure 1: Forward search on the small dataset.

Figure 1 shows how forward search works on the small dataset. The algorithm starts with no features and adds one feature at a time. Accuracy increases quickly after adding feature 2 and reaches the best result with features 2, 3. After adding more features, the accuracy goes down, which means these extra features probably add noise or are irrelevant.

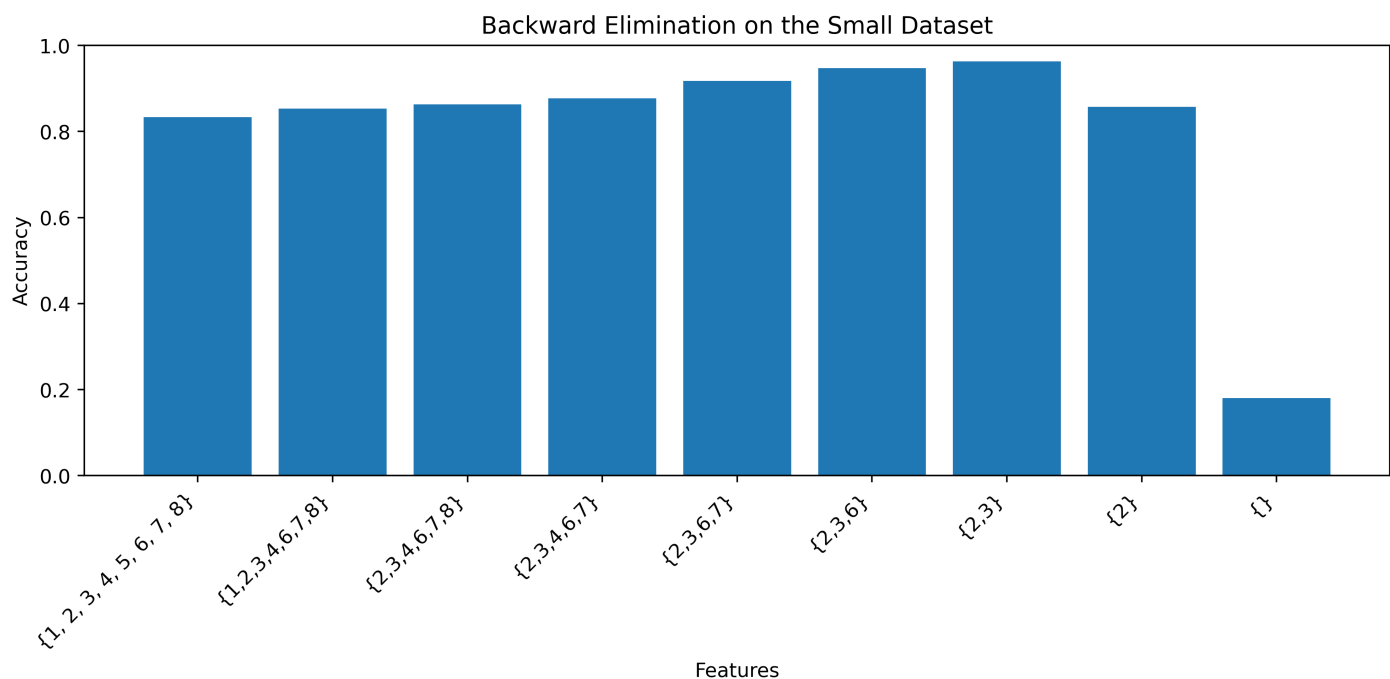


Figure 2: Backward elimination on the small dataset.

Figure 2 shows how backward elimination works on the small dataset. The algorithm starts with all features and removes one feature at a time. Accuracy improves as some unnecessary features are removed. The best result is again reached with features 2, 3. When the algorithm removes more features after that, accuracy drops.

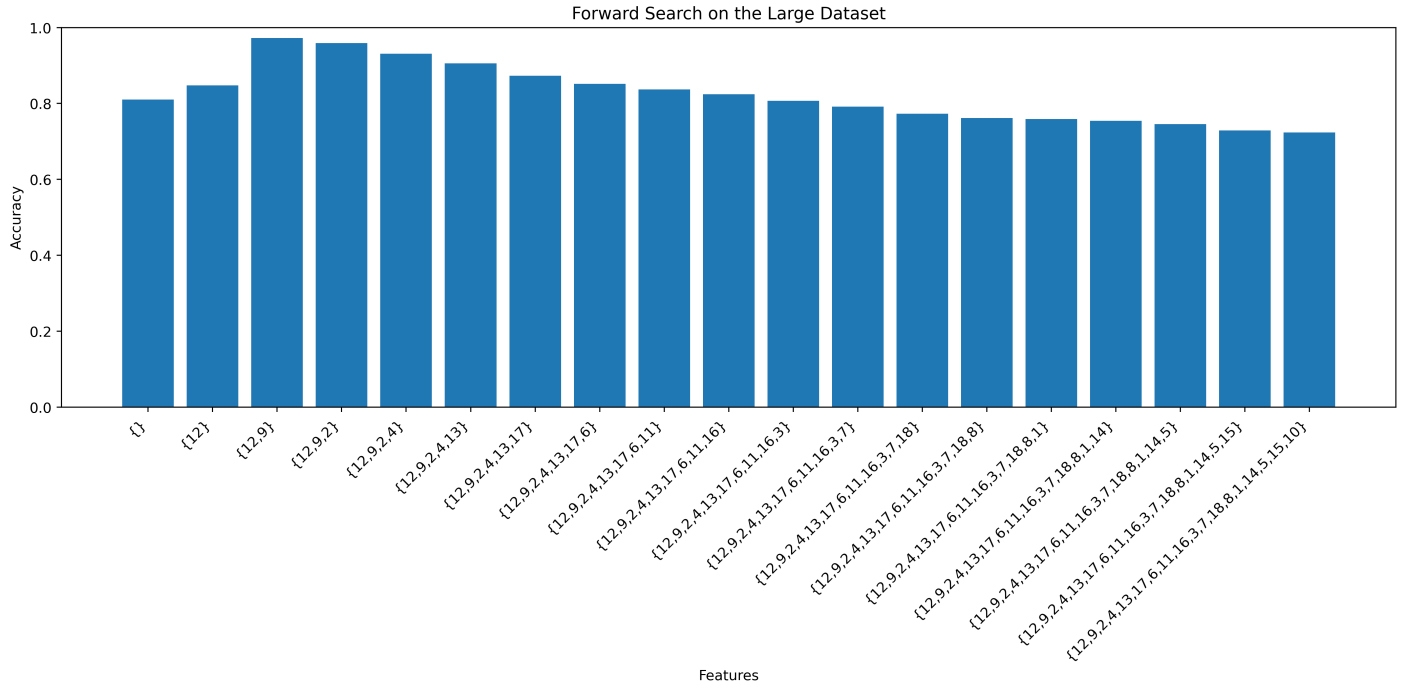


Figure 3: Forward search on the large dataset.

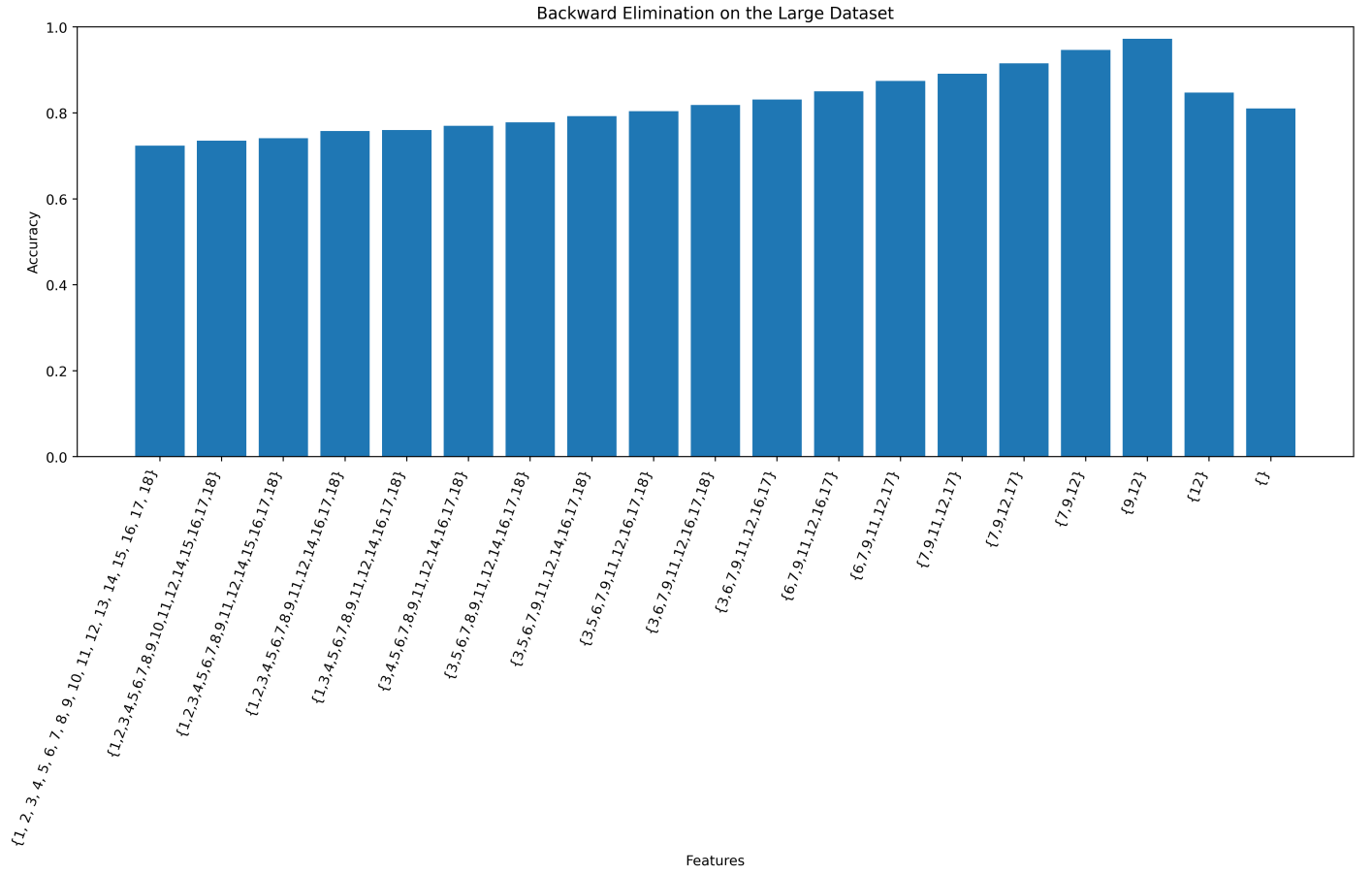


Figure 4: Backward elimination on the large dataset.

Figures 3 and 4 show how both methods work on the large dataset. In both cases, the accuracy reaches its highest value when the feature subset is {9, 12}. After additional features are added in forward search, or after more features are removed in backward elimination, the accuracy decreases.

For both datasets, feature selection significantly improved the accuracy, which means some of the original features could introduce noise or were irrelevant. Noisy or irrelevant features may decrease the quality of the model because the Nearest Neighbor algorithm estimates the distance between all features, so the final distance increases, making close features less close.

It is important to note that these best feature subsets were found for 1NN using the leave-one-out accuracy. However, this does not mean that the same feature subset will also be the best for a Decision Tree or a Random Forest because these models work differently. A Decision Tree separates classes using the threshold values, while a Random Forest tries to find features that improve the performance of an array of trees. Therefore, feature selection results also depend on the model.

Feature Selection on Real Dataset

To run feature selection on real data, we used the UCI Wine dataset [3]. The dataset has 13 features and 178 instances of wine made from 3 cultivars of grape grown in one Italian region.

Search Algorithm	Accuracy on All Features	Best Feature Subset	Accuracy on Best Subset
Forward Selection	0.95	{1, 2, 5, 7, 10, 11, 12, 13}	0.99
Backward Elimination	0.95	{1, 2, 4, 7, 8, 9, 10, 12, 13}	0.98
Intersection of the selected features	0.95	{1, 2, 7, 10, 12, 13}	0.95

Table 2: Feature selection results on the UCI Wine dataset using 1-nearest neighbor with leave-one-out evaluation.

Feature	Forward Selection	Backward Elimination
Alcohol	✓	✓
Malic acid	✓	✓
Ash		
Alcalinity of ash		✓
Magnesium	✓	
Total phenols		
Flavanoids	✓	✓
Nonflavanoid phenols		✓
Proanthocyanins		✓
Color intensity	✓	✓
Hue	✓	
OD280/OD315 of diluted wines	✓	✓
Proline	✓	✓

Table 3: Features selected by forward selection and backward elimination.

Table 2 shows the accuracy results on the UCI Wine dataset. Using all 13 features, 1NN classifier achieved an accuracy of 0.95. After feature selection, both algorithms improved this result. Forward selection achieved the best accuracy of 0.99, while backward elimination achieved 0.98. This means that removing some less useful features helped the classifier perform better. We also tested the intersection of the forward and backward selected features. This subset could not help improve the accuracy. Perhaps the intersection is too small, so some important features are not considered. It is possible that features 11 and 5 from the forward search set or features 4, 8, or 9 from the backward search set help 1NN separate classes better, they just did not appear in the intersection.

Table 3 shows which features were selected by each algorithm. Several features were chosen by both methods, these features seem to be the most stable because they were useful in both search methods. At the same time, some features were selected by only one algorithm, which shows that forward and backward search can find different useful combinations of features.

Let’s analyze each of the features:

- Alcohol. The concentration of alcohol differs between different kinds of wine, as different grapes have a specific amount of sugars, while sugar directly influences the alcohol concentration because it is processed into ethanol.

Apart from grapes, climate also influences the amounts of sugar: the sugar concentration increases faster in a warm climate. For this reason, for instance, California Cabernet usually has higher alcohol concentration than Cabernet from France. However, the data only contains wine from the same region, so in this case, it only helps us distinguish the grapes. Both algorithms found this feature one of the most relevant.

- **Malic acid.** Malic acid is the acidity, a natural characteristic of a grape. Some grapes are more acid, while others have a lower level of acidity. Soil and climate also influence this feature, but it is defined by the grape in the first place. This is probably why this feature was selected by both searches.
- **Ash.** It's the amount of minerals in wine [4]. Different grape types accumulate minerals differently, which is why wines differ in this characteristic. However, the main source of these minerals is still the soil. Since all the wines in the dataset come from the same region, both algorithms did not consider this feature as one of the most important ones.
- **Alcalinity of ash.** While ash tells us about the amount of minerals in wine, the alcalinity of ash is a chemical characteristics of these minerals [5]. By definition, it conveys more information than the ash feature, so alcalinity of ash may seem more important than the amount of minerals. The amount of minerals may be very similar in different types of grape, but grapes may contain different minerals. However, only backward elimination included this feature to the final features subset.
- **Magnesium.** This feature describes the amount of magnesium in wine. Wines in the dataset are produced from different grape, so they may differ not only in acidity, but also in mineral contents. Only forward search included this feature to the final feature subset. Although magnesium may seem irrelevant, it may be a strong feature in combination with other features that describe chemical profile of a wine.
- **Total phenols.** This feature tells about the overall amount of phenols in wine. Since phenols are a large group of chemical compounds, this feature may be too general to help separate the classes effectively. Moreover, the dataset contains more specific phenol features, such as flavanoids, so this feature may create redundancy. None of the algorithms selected this feature.
- **Flavanoids.** Flavanoids are antioxidants that can be found in many products, including grapes and other berries. They influence wine's texture, flavor, and color. Both algorithms found this feature closely related to the wine class and included it to the final subset.
- **Nonflavanoid phenols.** These are phenols that are not considered flavanoids. Backward elimination selected this feature, which suggests it may be useful for distinguishing classes. However, forward selection did not choose it, so its usefulness may depend on other features chosen by an algorithm. This feature more specific than the total amount of phenols, but it still may not have enough importance for being chosen by both methods.
- **Proanthocyanins.** This feature measures one specific group of polyphenolic compounds in wine. [6] shows that different grape cultivars can have different proanthocyanins profiles, and proanthocyanins are related to tannicity in red wines, which makes the taste dry and bitter. Because of this, this feature may help separate different wine classes. In our results, backward elimination selected this feature, so it probably gave some useful information for the classifier, though the forward method did not choose it.
- **Color intensity.** This feature measures how strong the wine color is. It may also be connected to flavanoids because flavanoids affect color, but color is a more general feature. Since both color intensity and flavanoids were selected by both algorithms, which suggests that color-related chemical properties may indeed help a model distinguish classes better.
- **Hue.** It describes the shade of the color, while color intensity describes how strong the color is. Forward selection selected this feature, but backward elimination did not keep it. Since color intensity and flavanoids were selected by both algorithms, hue was probably not useful in backward elimination because it describes color, but it could be useful in the feature combination found by forward selection with the features that backward method did not choose. In the subset chosen by the backward method, other features such as nonflavanoid phenols or alkalinity of ash may have provided enough additional information, so hue was less important in comparison to its important in the forward selected subset.
- **OD280/OD315 of diluted wines.** This is an optical density measured at wavelength either 280 or 315 nanometers [7]. The relation of these two density values is also based on ultraviolet light absorption. A study [8] on red wines

shows that UV spectroscopy can be used to analyze phenolic substances, which are related to astringency. Because phenolic composition can differ between wines, this feature may help distinguish wine classes. Both methods include this feature in the final feature set.

- Proline. This is an amino acid that is found in grapes and consequently in wine. Both methods chose this feature probably because this is the only amino acid. Most other features describe color, acidity or minerals level, while proline adds an additional information to a feature set.

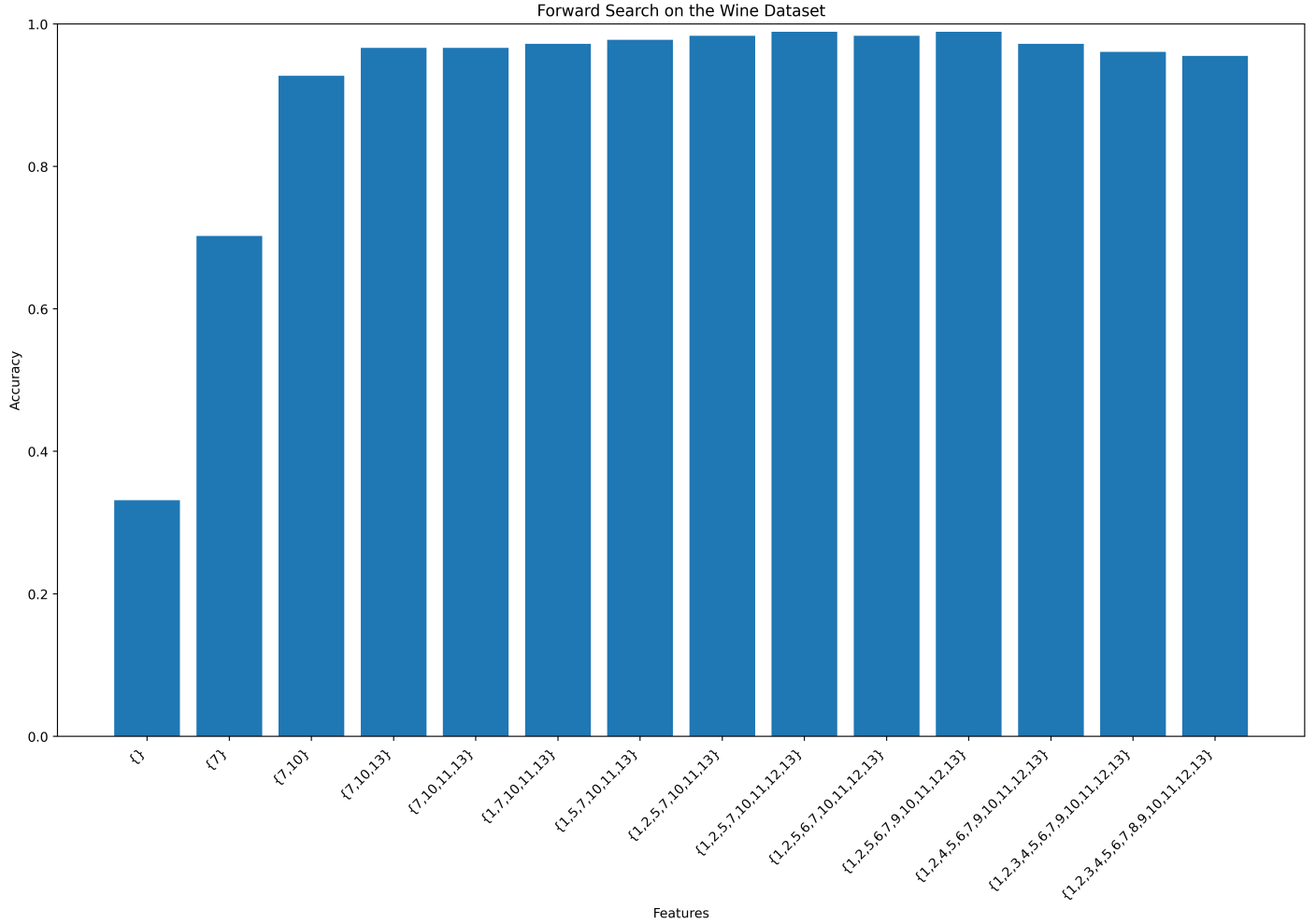


Figure 5: Forward search on the UCI Wine dataset.

Figure 5 shows that accuracy increases quickly when the first important features are added. The best result is reached with the subset $\{7, 10, 13, 11, 1, 5, 2, 12\}$, with an accuracy of about 0.98. After this, adding more features does not improve the result decreases the accuracy.

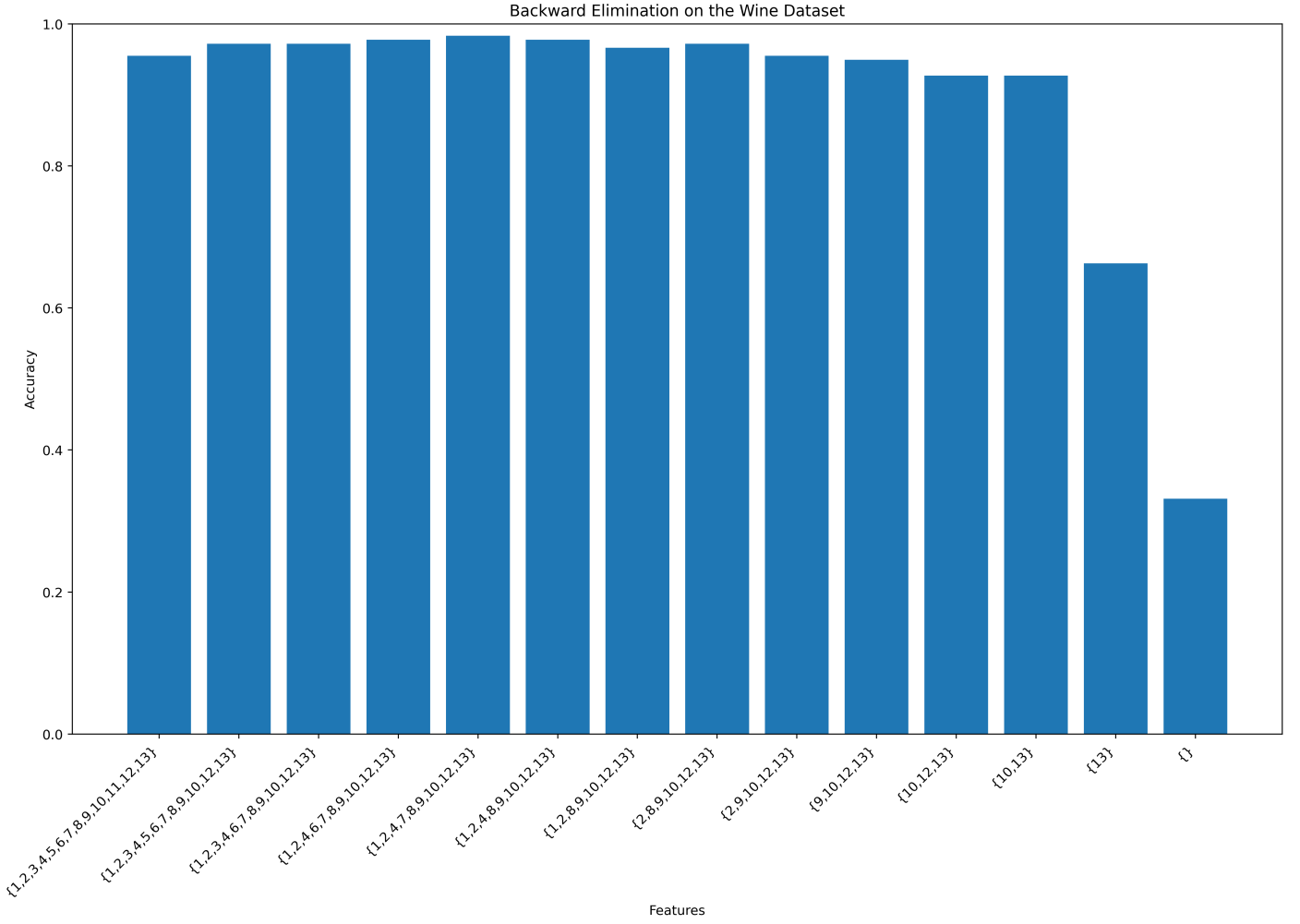


Figure 6: Backward elimination on the UCI Wine dataset.

Figure 6 shows that removing some features also improves the accuracy compared to using all features. The best result is reached with the subset $\{1, 2, 4, 7, 8, 9, 10, 12, 13\}$, with an accuracy of about 0.98. After more features are removed, accuracy decreases.

We can conclude that the selected features have a reasonable interpretation. Several features selected by both algorithms describe important properties of wine. The results also show that the two search algorithms do not choose the same features because they use different search paths. The forward methods starts with an empty set and adds features sequentially, while backward elimination starts with all features and then removes some of them. The intersection of the two selected subsets did not improve accuracy. Some features selected by only one algorithm may still add useful information, since very often it is much more important to look at the combination of features rather than on one feature.

Ideally, the analysis should start with defining the task, choosing the metric, and selecting the model. After that, we can perform feature selection. It is important because the best feature subset depends on the model and the metric. The features selected in this project worked well for 1-nearest neighbor with leave-one-out accuracy, but another classifier could select a different subset. The code for this part can be found at [9].

Dendrogram Clustering

In this part, we run hierarchical clustering analysis and display the dendrogram on dataset with 100 cities [10] to identify which cities are similar in terms of air temperature and wind speed. This analysis may help identify short-term weather patterns, such as heating or cooling demand. The dataset has the following columns: city, temperature (Celsius), wind speed (m/s), latitude, longitude, description, and country. Weather features describe weather conditions in each city on

April 28th, 2024. Since the dataset contains observations from only one day, the results should not be interpreted as evidence of long-term weather similarities. Although one day data is not enough for a thorough analysis and making any solid statistical conclusions about regional climate profiles, it can still reveal meaningful patterns in weather conditions. Most cities are expected to form clear clusters, while cities with noticeable different weather conditions may appear as outliers on a dendrogram. However, any possible outliers should not be connected to the weather anomalies for the following two reasons: 1. cluster organization depends on the linkage method, and 2. the data contains only one observation per city and does not include a time series, which would show how weather changes over time in each of the cities. Therefore, the main focus should be on identifying stable similarities that appear across different linkage methods, rather than trying to explain individual outliers.

Categorical variables were excluded because the clustering analysis was focused on grouping cities based on their weather characteristics. For the same reason, latitude and longitude were also removed. If we included geographical features, the algorithm would merge cities into clusters based on their geographical proximity, not weather similarities.

Dendrogram is a visualization that shows how objects can be grouped in clusters based on their similarity. This visualization looks as a tree, where leafs represent objects that are merged step by step according to a selected distance metric and linkage method. Branches define the unions of objects, and the length of a branch defined the dissimilarity between the clusters. So, objects that are connected at lower levels are more similar than objects that are connected at higher levels. There are several linkage methods that can be used to define the distance between clusters [11]. Some of them are:

- Single linkage, also known as the Nearest Point Algorithm, uses the shortest distance between clusters;
- Complete linkage, also known as the Farthest Point Algorithm or Voor Hees Algorithm. uses the largest distance;
- Average linkage, also known as the UPGMA algorithm, uses the average distance;
- Centroid linkage, also known as the UPGMC algorithm, uses the distance between cluster centers;
- Ward linkage estimates the variance in each cluster, thus making the objects as similar as possible.

In this task, we compared two linkage methods on normalized cities dataset. Figure 7 represents the dendrogram with the Ward linkage method, and Figure 8 represents the dendrogram with the Average linkage method.

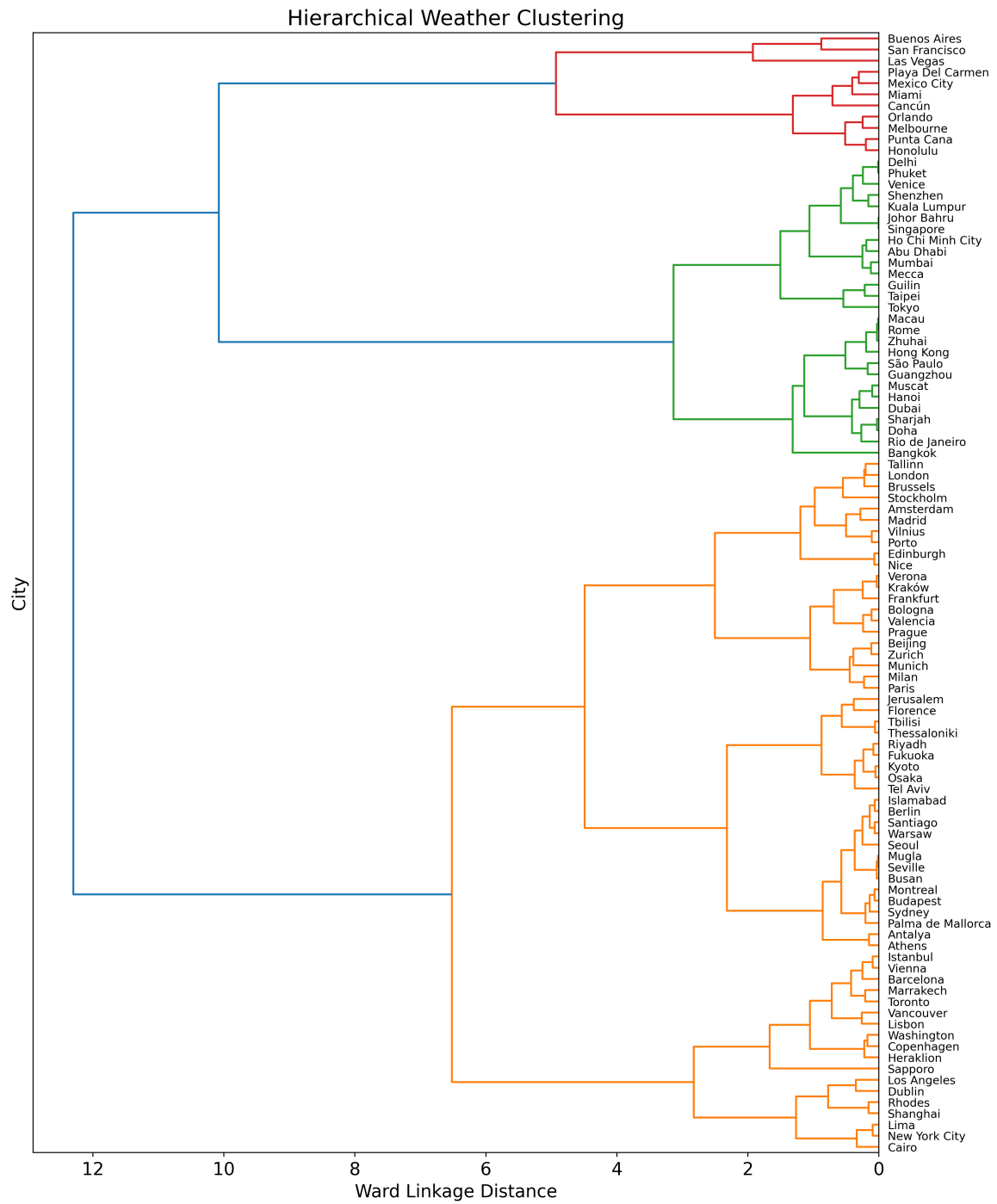


Figure 7: Dendrogram with Ward linkage method.

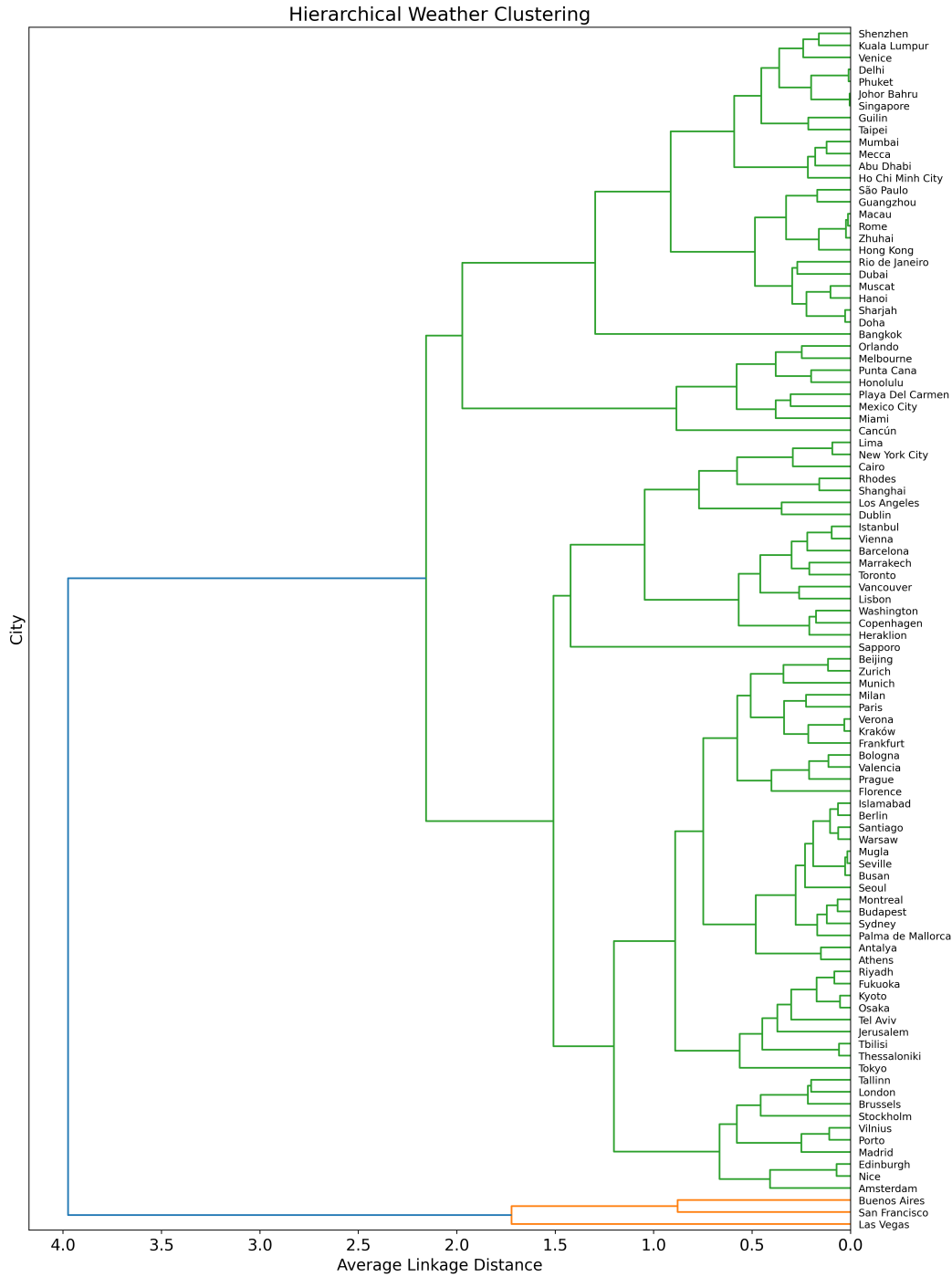


Figure 8: Dendrogram with Average linkage method.

Some results show that clustering captured some meaningful weather similarities. Moreover, the similarities were stable between the 2 linkage methods we tested. For example, Kyoto, Osaka, and Tokyo are grouped closely together on both charts. Kyoto and Osaka were merged very fast, with Tokyo was merged later. Since these cities are located in Japan and share similar climatic conditions, their proximity in the dendrogram looks reasonable. Similar patterns can also be observed for European cities as well. Zurich and Munich are merged at a very small linkage distance, and they are a part a larger group that also includes Milan and Paris. This is also plausible because the cities are located in Europe, and we know the climatic zone in Europe is more or less similar across different countries. Another pair, London and Edinburgh, were also placed in the same subgroup in both dendrograms, and we know these cities are located on the same island and probably share very similar climatic conditions. Amsterdam is also close to London on both charts, which makes sense,

as these countries are separated by the North Sea. At the same time, we see that this group also has such warm cities as Porto or Nice, but it does not necessarily mean that the clustering is incorrect or random. Although these cities are from different countries and climate subregions, they are all located near a sea or an ocean, so they may have similar maritime climate. Therefore, their grouping may reflect genuine similarities in temperature and wind observed on that particular day. In contrast, Buenos Aires with San Francisco and Las Vegas form a separate cluster in the Average-linkage dendrogram but not in the Ward dendrogram, which supports the idea that the clusters structure may depend on the chosen linkage method.

At a larger scale, both dendrograms can reveal some reasonable patterns. In the Ward dendrogram, the green cluster represents cities from subtropical, and desert regions. In the Average-linkage dendrogram, although there is only one large cluster, we can notice that cities are grouped in a similar way, even within one cluster. Although we excluded the geographical features, we can see that the clustering was able to group cities that turned out to be close to each other.

Overall, these observations increase confidence that the clustering captures meaningful structure in the data rather than random patterns. We can conclude that the Ward method helps to produce more clear and separable groups. It is important to note that we compared only two linkage methods, but ideally all common linkage methods should be compared. This would help determine which method separates the cities more clearly and produces the most meaningful cluster profile.

Conclusion

In this project, we applied the forward selection and backward elimination algorithms to find the most important features in two synthetic and one real dataset. The selected feature subsets helped 1-nearest neighbor classifier achieve a higher accuracy in comparison to the accuracy it yielded on all original features. We also used hierarchical clustering and displayed dendrograms to explore the relationships between cities based on weather characteristics.

The results demonstrate that feature selection not only decreases the number of features, which reduces computational cost, but also may improve the model's performance. However, feature selection was evaluated using a 1-nearest neighbor classifier. 3-nearest neighbors or 5-nearest neighbors are usually more preferable, as they are less sensitive to noise. In addition, it would also be interesting to apply feature selection using other classification algorithms, as different models may select different features. Comparing the result would help us understand whether the selected features are the most useful in general or whether they are useful for a specific algorithm only. In the clustering part, the results showed that although two dendrograms are visually different, they group cities into similar clusters. However, only two linkage methods were compared and only two weather features were used, so it would be useful to parse more data for these cities and test other common linkages to see which of them helps separate cities better.

References

- [1] Synthetic datasets. https://github.com/SreejaPagireddy/CS205_-Dendrogram_Features/tree/main/synthetic_datasets.
- [2] Feature selection code for Winter 2025 CS170. First created on Feb 25, 2025. <https://github.com/SreejaPagireddy/NearestNeighborCS170/blob/main/dataset.py>.
- [3] UCI Wine dataset. <https://archive.ics.uci.edu/dataset/109/wine>.
- [4] Compendium of International Methods of Wine and Must Analysis. [https://www.oiv.int/standards/compendium-of-international-methods-of-wine-and-must-analysis/annex-a-methods-of-analysis-of-wines-and-musts/section-2-physical-analysis/ash-\(type-i\)](https://www.oiv.int/standards/compendium-of-international-methods-of-wine-and-must-analysis/annex-a-methods-of-analysis-of-wines-and-musts/section-2-physical-analysis/ash-(type-i)).
- [5] Determination of ash content and ash alkalinity. <https://prometheusprotocols.net/function/tissue-chemistry/structural-compounds/determination-of-ash-content-and-ash-alkalinity/>.
- [6] Arapitsas, Panagiotis, et al. "Decoding the Proanthocyanins Profile of Italian Red Wines". *Beverages*, vol. 8, no. 4, 2022, p. 76. <https://doi.org/10.3390/beverages8040076>.
- [7] Optical density and absorbance measurements. <https://www.bmglabtech.com/en/blog/optical-density-for-absorbance-assays/>

- [8] J.C. Boulet, M.A. Ducasse, V. Cheynier, "Ultraviolet Spectroscopy Study of Phenolic Substances and Other Major Compounds in Red Wines: Relationship between Astringency and The Concentration of Phenolic Substances". <https://doi.org/10.1111/ajgw.12265>.
- [9] Feature selection on UCI Wine dataset. https://github.com/SreejaPagireddy/CS205_-Dendrogram_Features/blob/main/Feature_search_synthetic_and_wine_dataset.ipynb
- [10] Top 100 Cities Weather Dataset. <https://www.kaggle.com/danishammar/top-100-cities-weather-dataset>.
- [11] SciPy. linkage. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.linkage.html#scipy.cluster.hierarchy.linkage>.
- [12] Pandas Documentation. https://pandas.pydata.org/docs/user_guide/index.html#user-guide.
- [13] SciPy Dendrogram Documentation. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.dendrogram.html>.
- [14] Medium Article on Selecting Columns in Pandas. <https://medium.com/@whyamit101/pandas-only-keep-certain-columns-7710d6f71a56>.
- [15] The Python Graph Gallery. Basic Dendrogram. <https://python-graph-gallery.com/400-basic-dendrogram/>
- [16] Lecture slides.